Enhancing Code Performance: Time Complexity (with Generative AI)

Understanding Time Complexity

Time Complexity measures how the execution time of an algorithm grows with input size.

It is expressed using **Big-O notation**.

Common Time Complexities

Big-O	Name	Example
O(1)	Constant	Access array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Single loop
O(n log n)	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Nested loops
$O(2^n)$	Exponential	Recursive Fibonacci

Why Time Complexity Matters

- Faster execution
- Better scalability
- Lower server cost
- Improved user experience

Optimizing Time Complexity with Generative AI

Generative AI tools (GitHub Copilot, ChatGPT):

- Analyze algorithms
- Detect inefficient loops
- Suggest better data structures
- Replace brute-force with optimized approaches

How Generative AI Improves Time Complexity

Action	AI Contribution
Identify nested loops	Yes
Suggest hashing	Yes
Convert linear search → binary search	Yes
Suggest sorting or indexing	Yes

Example 1: From $O(n^2)$ → $O(n)$

Before (Quadratic)

```
def find_duplicates(nums):
    duplicates = []
    for i in nums:
        for j in nums:
            if i == j and i not in duplicates:
                duplicates.append(i)
    return duplicates
```

After (AI Optimized – Linear)

```
def find_duplicates(nums):
    seen = set()
    duplicates = set()
    for n in nums:
        if n in seen:
            duplicates.add(n)
        else:
            seen.add(n)
    return list(duplicates)
```

Example 2: Linear Search → Binary Search

Before (O(n))

```
def find_number(nums, target):
    for n in nums:
        if n == target:
            return True
    return False
```

After (AI Suggestion – O(log n))

```
import bisect

def find_number(nums, target):
    nums.sort()
    index = bisect.bisect_left(nums, target)
    return index < len(nums) and nums[index] == target
```

Example 3: Inefficient String Building

Before ($O(n^2)$)

```
result = ""
for s in words:
    result += s
```

After (O(n))

```
result = "".join(words)
```

Time Complexity with Generative AI: Demo Workflow

1. Paste code into IDE
2. Add comment:

```
# Optimize time complexity
```

3. Copilot suggests faster algorithm
4. Developer reviews and tests

Benefits of Using GenAI for Time Optimization

- Saves analysis time
- Provides alternative algorithms
- Improves code quality

Limitations

- AI may not know business constraints
- Might suggest memory-heavy solutions
- Requires human validation

Quick Check

Q1. What does $O(n^2)$ mean?

Execution time grows with square of input

Q2. True/False

Binary search is faster than linear search.
True

Q3. Which tool can help optimize time complexity?

A) Calculator
B) GitHub Copilot
C) Paint

B

Summary

Generative AI helps developers:

✔ Identify inefficient code
✔ Suggest better algorithms
✔ Reduce time complexity

But **developer expertise remains essential**.
